Domain Intelligence Layer: Concept, Architecture, Build Plan and Go-to-Market

Oct 12, 2026 · @Anurag

1. Executive summary

The Domain Intelligence Layer (DIL) is a reusable layer that sits between an LLM and a company's data and makes the system reason like a specialist in one industry. RAG and databases supply facts; the DIL supplies judgment: the vocabulary, rules, expert workflows, guardrails and tests that a good analyst carries in their head. We recommend starting with banking credit and compliance operations, using a policy-gated orchestration pattern, and proving value with a measured 8-week pilot.

The idea in one paragraph. Most enterprise LLM projects today connect a model to documents (RAG) and to structured data (database or APIs). That gives fluent answers built on correct facts, but the reasoning is generic: the model does not know which rule applies, which steps an expert would follow, or what it must never say. The DIL adds that missing block. It can work alongside RAG and the database, wrap them and decide when and how they are used, or work on its own when no data is connected.

Why it matters. In regulated industries a fluent but subtly wrong answer is worse than no answer. Buyers need accuracy they can measure, rules they can audit and a faster path than building everything in-house with scarce domain experts. The DIL packages exactly those three things.

Five decisions this document asks you to make

#	Decision	Recommendation	Where
1	What to call it	Domain Intelligence Layer (DIL); use Domain Expertise Layer for business audiences	Section 3
2	First domain	Banking: credit documentation, KYC/AML review, regulatory Q&A	Sections 6 and 7
3	Integration pattern	Policy-gated orchestration (DIL decides when to retrieve, query and validate), with a plain-prompt mode for pilots	Section 9

#	Decision	Recommendation	Where
4	Build or buy the plumbing	Buy or reuse managed RAG and guardrail components; build the domain content yourself, because that is the moat	Section 10
5	Business model	Annual domain-pack license plus implementation fee plus rule-update subscription	Sections 13 and 14

What this document is not. It is a framework and plan, not market research. Where it gives figures (timelines, savings, prices) they are labeled as illustrative assumptions to be replaced by your own pilot data. Vendor and product names are examples from general knowledge and should be checked for current availability before any purchasing decision.

2. The problem: facts are not judgment

Connecting an LLM to company data solves the facts problem but not the judgment problem. A model can quote the right policy paragraph and still apply it wrongly, skip a mandatory check or answer something it should have escalated.

2.1 How companies connect LLMs to data today

Three patterns dominate, and they are the three scenarios in the original diagram.

Scenario	How data reaches the LLM	Strength	Gap
1. LLM only	None; training knowledge only	Simple, cheap	No company facts; generic reasoning
2. LLM + database	Model generates a query or calls an API and reads structured results	Exact, current numbers	No understanding of what the numbers mean in context or policy
3. LLM + RAG pipeline + database	Documents are chunked, embedded and retrieved per question, combined with structured data	Grounded, citable answers over large unstructured content	Retrieval brings facts, not the expert procedure for using them

Other ways data reaches a model (data pasted into the prompt, fine-tuning, real-time feeds, user memory, knowledge graphs) exist, and Section 9 shows how the DIL combines with each of them.

2.2 Where plain RAG falls short in regulated domains

- **Right facts, wrong rule.** Retrieval returns the most similar passages, not the applicable ones. A superseded circular and its replacement can both be retrieved.
- **No process.** An analyst follows a sequence: identify the customer type, check the product rules, check limits, compute, document the reason. A single retrieval-then-answer step skips that sequence.
- **No refusal logic.** The model does not know what it must not say, such as personalized investment advice, unverified legal conclusions or disclosure of another customer's data.
- **Arithmetic and thresholds.** LLMs approximate numbers. Ratios, limits and dates must be computed by code and checked against rules.
- **Inconsistency.** Two staff asking the same question can get differently structured answers, which auditors dislike.
- **No proof of accuracy.** Without an expert-graded test set, nobody can say how often the system is right, so risk teams cannot approve it.

2.3 The missing block

What is missing is a layer that is neither retrieval nor storage. It encodes how an expert in the field thinks and what the organization is allowed to do. That layer is the Domain Intelligence Layer.

	Facts layer (RAG, database)	Judgment layer (DIL)
Question it answers	What does our data say?	What should we do with it, and is it allowed?
Changes when	Documents and records change	Regulations, products and policy change
Owned by	Data and engineering teams	Domain experts, compliance and risk, with engineering
Failure looks like	Missing or stale information	Plausible but wrong or non-compliant answer

3. Definition and naming

The Domain Intelligence Layer is the set of domain rules, terminology, expert workflows, guardrails and tests that make an LLM reason like a specialist, with or without connected data. It is a layer of behavior and control, not a store of data.

3.1 One-line definition

"Domain rules, terminology and expert workflows that make the LLM reason like a specialist, with or without connected data."

3.2 What it is and what it is not

Concept	What it does	How the DIL differs
RAG pipeline	Retrieves relevant document passages	DIL decides whether, what and how to retrieve, and checks the result against rules

Concept	What it does	How the DIL differs
Database / Text-to-SQL	Returns exact records	DIL knows which tables matter for the task and validates the numbers
Fine-tuning	Changes model weights for style or skills	DIL changes behavior through rules, workflows and checks that can be updated in days, not retrained
Semantic layer	Defines business metrics and terms for analytics	DIL includes it as one component, then adds rules, workflows and guardrails
Guardrails	Block unsafe inputs and outputs	DIL includes them as one component, then adds positive guidance on how to do the task well
Prompt template	Fixes instructions for one task	DIL is a governed, versioned, tested library of such templates plus rules and validators
Agent framework	Runs multi-step tool use	DIL is the domain content an agent framework executes

3.3 Naming options

The concept already travels under several names. Which one to use depends on the audience.

Name	Emphasis	Best audience
Domain Intelligence Layer	Whole concept, product-like	Executives, buyers, investors
Domain Expertise Layer	Specialist thinking	Business and operations teams
Semantic and Policy Layer	Definitions plus rules	Data and platform engineers

Name	Emphasis	Best audience
Knowledge or Ontology Layer	Concepts and relationships	Knowledge-graph teams
Domain Skills / Domain Agents	Packaged workflows	Agent and platform teams
Context Engineering Layer	Everything shaping model behavior	AI engineers

Recommendation: use "Domain Intelligence Layer" in diagrams and sales material, and describe its parts with the technical names above. Pair each domain with a name, for example "Banking DIL" or "Insurance DIL".

3.4 The three ways it can sit in the scenario diagram

In the original Scenario 3 (LLM, RAG pipeline, database, use case), the new block can be added three ways. Section 9 expands these into seven integration patterns.

- **Alongside:** DIL is one more component the LLM calls, next to RAG and the database.
- **Wrapping:** DIL surrounds RAG and the database and controls them: when to retrieve, which source to use, how to validate.
- **Standalone:** DIL works with no data connected, still improving reasoning through rules, terminology and workflows.

4. Components of the layer

The DIL has six components. Components 1 to 5 create the expertise; component 6 connects it to the company's systems. Each is described below with a banking example, the artifact a team actually produces, and who owns it.

4.1 Terminology and ontology

What it is. A governed glossary of domain terms, their definitions, synonyms and relationships.

Why it matters. LLMs treat "NPA", "substandard asset" and "non-performing" as probably related. A bank needs them to be exactly defined, with the right classification thresholds, and linked to the right products and regulations.

Banking example. NPA, CASA, LTV, DSCR, Tier 1 capital, KYC tiers, PEP, and how loan types, collateral types and risk grades relate.

Artifact. A glossary file or graph (for example JSON, RDF or a property graph), with synonyms and a definition source for each term.

Owner. Business analyst plus a subject-matter expert (SME), reviewed by compliance.

4.2 Rules and regulation logic

What it is. Business and regulatory rules expressed as checks a program can run, not as paragraphs the model may or may not follow.

Banking example. Loan-to-value limits by product, exposure limits, KYC triggers by transaction type and amount, sanctions-screening hit handling, escalation thresholds.

How to express it. A rules engine or decision tables (for example DMN, Drools, OPA or plain code), each rule carrying an ID, source reference, effective date and owner. The LLM extracts inputs and explains results; the engine decides.

Artifact. A versioned rule repository with unit tests per rule.

Owner. Compliance and risk, with engineering implementing.

4.3 Expert workflows (skills and agents)

What it is. Step-by-step procedures for common tasks, written the way an experienced analyst works.

Banking example: credit memo drafting. Identify borrower type, collect financial statements, compute ratios, check policy limits, compare with peer benchmarks, list risks, draft the memo with citations, flag items for a human reviewer.

How to build it. As a workflow graph (for example in LangGraph or a similar framework) or as structured instructions with tool calls. Every step names the tool it may use and the output it must produce.

Artifact. A workflow specification per task, with inputs, steps, tools, outputs, escalation points and a test set.

Owner. Process owners and senior analysts, with engineering.

4.4 Guardrails and validators

What it is. Checks that run before and after the model answers.

Check type	Banking example
Input filter	Block requests to reveal another customer's data or to bypass a control
PII handling	Mask account numbers, national IDs and phone numbers in prompts and logs
Scope control	Refuse personalized investment advice or legal conclusions
Numeric validation	Recompute every ratio and total in code and compare with the draft
Citation check	Every factual claim must point to a retrieved source
Output policy	Required disclaimers, tone and format for customer-facing text
Escalation	Route low-confidence or high-risk cases to a human

Artifact. A policy configuration plus validator code and tests.

Owner. Risk and security, with engineering.

4.5 Evaluation sets

What it is. A collection of real questions and tasks with expert-approved answers or acceptance criteria. This is the component most teams skip and the one that proves accuracy to a buyer.

Banking example. 300 to 500 questions drawn from real analyst work, tagged by task, difficulty and rule, including adversarial cases such as outdated circulars, conflicting documents and attempts to bypass controls.

Artifact. A versioned test set plus an automated runner and score history.

Owner. SMEs for content, engineering for tooling.

4.6 Connectors

What it is. Ready integrations to the systems that supply facts and receive actions.

Banking example. Core banking system, loan origination system, CRM, document management, regulatory publication feeds, sanctions lists, credit bureau APIs.

Artifact. Connector code or configuration, with access scopes defined per role.

Owner. Engineering and IT, with security review.

4.7 Summary: what each component adds

Component	Adds	Without it
Terminology and ontology	Precise meaning	Misread terms, wrong retrieval
Rules logic	Correct decisions	Plausible but non-compliant answers
Expert workflows	Consistent process	Variable quality, skipped steps
Guardrails	Safety and scope control	Leaks, advice the bank must not give
Evaluation sets	Measurable accuracy	No basis for approval or improvement
Connectors	Access to real systems	A demo that never reaches production

5. Pros and cons

The DIL pays off where mistakes are costly and rules are dense; it is overhead where the task is simple and the stakes are low. Its main cost is expert time, not software.

5.1 Pros

Benefit	Why it happens	Who feels it
More accurate reasoning	Correct terms, applicable rules and the right procedure are applied	Analysts, customers
Consistency	Same workflow and format every time, regardless of who asks	Operations, auditors

Benefit	Why it happens	Who feels it
Compliance built in	Rules are checked on every answer, not left to the prompt	Compliance, risk
Safer output	Guardrails block unsafe advice, leaks and unverified numbers	Risk, security, legal
Reusable across use cases	One layer serves loans, fraud, support and reporting	Product, engineering
Model-independent	Rules and workflows live outside the model, so models can be swapped	CTO, procurement
Faster change	A rule update takes days; retraining a model takes weeks	Compliance, product
Auditability	Rule IDs, versions and citations show why an answer was given	Auditors, regulators
Works with no data	Terminology, workflows and guardrails improve output even when nothing is retrieved	Early pilots
Measurable	Evaluation sets turn "seems good" into a score	Executives, approvers

5.2 Cons and costs

Drawback	Detail	Mitigation
Needs real experts	SMEs must define rules, review outputs and grade tests; their time is scarce and expensive	Start with one narrow use case; use existing SOPs and checklists as raw material
Maintenance burden	Regulations, products and policies change; the layer goes stale without owners	Assign an owner and review date to every rule; subscribe to regulatory feeds
Over-restriction	Strict rules and guardrails can make answers stiff or cause needless refusals	Measure refusal rate in evaluation; tune thresholds with users

Drawback	Detail	Mitigation
Does not replace facts	Without RAG or a database the model still cannot know your policies or customers	Treat DIL as a complement to the facts layer, never a substitute
Added latency and cost	Extra steps (rule checks, validation, multiple model calls) slow responses and raise cost	Use rule engines and small models for checks; cache stable results
Integration effort	Connecting rules, workflows and systems takes engineering time	Reuse managed components; integrate one system at a time
False confidence	Users may over-trust an answer because it "passed the rules"	Show sources and confidence; keep a human reviewer on high-risk steps
Rule errors scale	A wrong rule is applied to every case until found	Unit tests per rule, staged rollout, monitoring of outcomes
Vendor competition	Large platforms are shipping their own domain packs	Differentiate on depth, local regulation and proven accuracy

5.3 When not to build it

- The task is low-stakes and rules are few, such as internal brainstorming or drafting marketing copy.
- There are no domain experts available to own the content, even part-time.
- The data and processes are too unstable to codify yet; stabilize the process first.
- Plain RAG already scores well enough on a real evaluation set. Measure before adding layers.

5.4 A fair comparison with the alternatives

Approach	Time to first value	Accuracy in rule-heavy tasks	Ongoing effort	Best for
Prompt only	Days	Low	Low	Experiments
Plain RAG	Weeks	Medium for lookup, low for judgment	Medium	Document Q&A

Approach	Time to first value	Accuracy in rule-heavy tasks	Ongoing effort	Best for
Fine-tuning	Months	Medium to high for style and format, uneven for changing rules	High	Stable tone and format
DIL on top of RAG and database	Weeks to a few months	High where rules are explicit	Medium to high (expert time)	Regulated, rule-heavy work

The ratings are qualitative judgments for planning, not measured benchmarks. Replace them with your own evaluation results after the pilot.

6. Which domain to start with

Start with banking credit and compliance operations: it scores highest on rule density, cost of errors and willingness to pay. Insurance claims and underwriting is the closest second and the natural expansion.

6.1 Selection criteria

Each domain is scored from 1 to 5 on six criteria, where 5 is always better for a new entrant.

Criterion	What a 5 means
Value of accuracy	Errors are costly (fines, losses, harm), so buyers pay for proof of correctness
Rule density	Work follows explicit, documentable rules, which is where a DIL adds most
Data readiness	Data is digital, accessible and reasonably clean
Budget	Buyers have AI budgets and a history of buying software
Sales speed	Procurement and security review move in months, not years

Criterion	What a 5 means
Low competition	Few specialized, well-funded rivals already serve the use case

6.2 Scoring

The scores below are planning judgments from general industry knowledge, not measured data. Re-score them with 5 to 10 customer conversations before committing.

Domain	Accuracy value	Rule density	Data readiness	Budget	Sales speed	Low competition	Total (max 30)
Banking: credit and compliance operations	5	5	4	5	2	3	24
Insurance: claims and underwriting	4	4	3	4	3	3	21
Healthcare administration: coding, prior authorization	5	4	2	4	2	2	19
Legal and contracts	4	3	4	4	3	1	19
Retail and e-commerce	3	2	5	3	4	2	19
Telecom and customer support	3	3	4	3	4	2	19
HR and recruitment compliance	3	3	4	2	4	3	19
Manufacturing and supply chain	4	3	2	3	2	4	18

6.3 Why banking wins, and its catch

- **Rules are explicit.** Regulations, credit policies and KYC procedures are written down, versioned and enforced, so they can be turned into checks.
- **Mistakes are expensive.** Compliance failures bring fines and reputational damage, so accuracy has a clear price.
- **Work is document-heavy.** Credit files, KYC packs and policy manuals suit RAG plus structured checks.
- **Buyers have budgets** and already pay for compliance technology.

- **The catch is sales speed.** Security reviews, model risk approval and vendor onboarding are slow. Plan for 6 to 12 months from first meeting to production and use pilots to stay inside budget limits that need no board approval.

6.4 Why not the others first

- **Healthcare:** strong need, but patient data access, clinical validation and liability slow everything, and data is often messy.
- **Legal:** dominated by well-funded specialist vendors; hard for a new entrant to stand out.
- **Retail and telecom:** fast to sell, but rules are thin and the value of accuracy is lower, so the DIL is harder to justify over plain RAG.
- **Manufacturing:** good low-competition space, but data readiness is poor and each plant differs.

6.5 Expansion path

1. Banking credit and compliance operations (first 12 months).
2. Adjacent banking functions: customer support, collections, treasury operations.
3. Insurance: claims triage and underwriting, reusing the rules engine, guardrails and evaluation tooling.
4. Other regulated verticals (healthcare administration, legal) using the same architecture with new domain content.

The architecture and tooling are the reusable part; the domain content is rebuilt for each vertical, which is why each new vertical is a new product pack, not a configuration change.

7. Banking deep dive

The best first wedge in banking is one document-heavy, rule-heavy workflow with a measurable cost per case, such as credit file review or KYC refresh. This section lists candidate use cases, the sources of rules, a worked example and what a pilot looks like.

7.1 Candidate use cases

Use case	Typical users	Main data	Rule intensity	Fit as first wedge
Credit file review and memo drafting	Credit analysts, relationship managers	Financial statements, bureau reports, policy manuals	High	Strong: clear output, clear time savings
KYC and customer due diligence refresh	Compliance operations	ID documents, ownership records, screening results	High	Strong: repetitive, auditable
AML alert triage	Financial crime teams	Transactions, alerts, customer profile	High	Medium: high stakes, needs model risk approval
Regulatory change impact analysis	Compliance, legal	Circulars, internal policies	Medium	Medium: valuable but harder to measure
Internal policy and procedure Q&A	Branch and operations staff	Policy manuals, circulars	Medium	Good low-risk entry; weak differentiation alone
Customer-facing support assistant	Customers, contact centers	Product data, account data	Medium	Later: higher conduct and reputational risk
Complaints handling	Customer service, compliance	Complaint records, policies	Medium	Medium

Recommended first wedge: credit file review and memo drafting, paired with internal policy Q&A as a low-risk entry point to build trust.

7.2 Where the rules come from

Source	Examples	How it enters the DIL
Regulator publications	Central bank circulars and master directions, Basel framework documents, FATF recommendations	Parsed into rule candidates; SMEs approve each rule
Internal credit and risk policy	Credit policy manual, delegation of authority, limits	Rule repository with owners
Product terms	Loan product sheets, fee schedules	Structured product catalog
Past decisions	Approved and rejected credit memos, with reasons	Evaluation set and example library
Analyst know-how	Checklists, spreadsheets, tacit rules gathered in interviews	Workflow steps and heuristics, reviewed by seniors

Exact regulations differ by country and change often. Each rule in the repository must carry its source, effective date and owner, and be re-checked on a schedule.

7.3 Worked example: credit file review

The workflow below shows how the layers cooperate on one task.

1. **Intake.** The analyst uploads the borrower's file. Connectors pull bureau data and prior exposure from internal systems. Guardrails mask sensitive fields in logs.
2. **Classify.** The DIL identifies borrower type (for example salaried, MSME, corporate) using the ontology, which selects the applicable policy and ratio set.
3. **Extract.** The LLM extracts financial statement figures into a structured form. A validator checks that totals reconcile and flags missing items.
4. **Compute.** Code, not the LLM, calculates ratios such as DSCR, leverage and LTV.
5. **Check rules.** The rules engine tests each ratio and attribute against policy limits and returns pass, fail or refer, with the rule ID for each.
6. **Retrieve context.** RAG fetches relevant policy clauses, sector guidance and similar past cases, filtered by product and effective date.
7. **Draft.** The LLM writes the memo in the bank's template, explaining rule outcomes and citing sources for every factual claim.

8. **Validate.** The numeric validator recomputes figures in the draft; the citation checker confirms each claim has a source; scope checks block disallowed language.
9. **Escalate.** Any failed hard rule, low-confidence extraction or policy conflict is routed to a senior reviewer with reasons attached.
10. **Log.** Inputs, rule versions, retrieved sources, outputs and reviewer decisions are stored for audit.

In this flow the LLM reads, explains and drafts; the rules engine and code decide and calculate; the DIL orchestrates.

7.4 What a pilot looks like

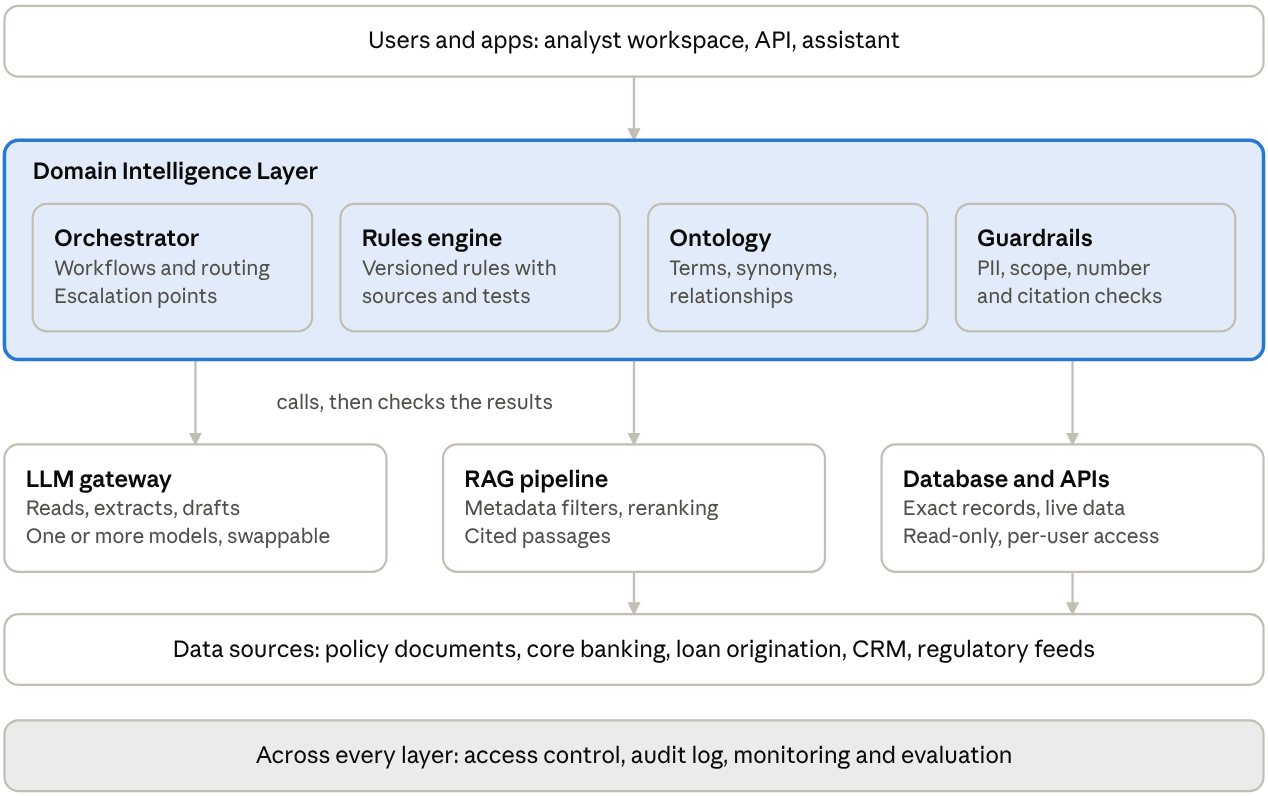
Item	Proposal
Scope	One credit product segment, one branch cluster or one team, 5 to 15 analysts
Duration	8 weeks
Baseline	Measure current time per file, error and rework rates, and reviewer comments on a sample of past files before starting
Success metrics	Analyst time per file, first-pass reviewer acceptance, rule-violation catch rate, factual error rate on the evaluation set
Control	Compare with a plain-RAG assistant on the same files to show the DIL's added value
Safeguards	Human approval on every output; no customer-facing use; sandboxed data copy or private cloud deployment
Exit criteria	Agreed numeric targets set with the customer before kickoff, for example a threshold improvement in time per file and a ceiling on factual error rate

The targets must be set with the customer from their own baseline. This document does not assume specific improvement numbers.

8. Reference architecture

Every request passes through the DIL, which decides which sources to call, runs the rules and checks, and returns an answer only after the checks pass.

The DIL sits between the user and every source of facts



reference architecture · 3 layers, 9 components

The orchestrator, rules engine, ontology and guardrails form the DIL. The LLM gateway, RAG pipeline and database are the language and facts layers it controls.

8.1 Component responsibilities

Component	Responsibility	Examples of technology	Owner
Orchestrator	Runs workflows, routes requests, calls tools, handles retries and escalation	LangGraph, Temporal, a custom state machine	Engineering

Component	Responsibility	Examples of technology	Owner
Rules engine	Evaluates rules and returns the outcome with the rule ID	OPA, Drools or a DMN engine, decision tables in code	Compliance with engineering
Ontology and glossary	Normalizes terms, maps synonyms, links products and regulations	Graph database or structured files	Business analysts and SMEs
Guardrails and validators	Input and output checks, PII masking, numeric and citation checks	NeMo Guardrails, Guardrails AI, cloud guardrail services, custom code	Risk with engineering
LLM gateway	Single entry to one or more models; routing, logging, redaction, cost control	Cloud model services, self-hosted models, API gateways	Platform team
RAG pipeline	Parsing, chunking, embedding, retrieval, reranking, citations	Managed RAG services, LlamaIndex, Haystack; parsers such as Docling or Unstructured	Data and ML
Database and APIs	Exact records and live data, read-only by default	Warehouse, operational databases, internal APIs	Data team
Identity and audit	User identity passed to every call; immutable logs	Enterprise single sign-on, secrets manager, log store	Security
Evaluation and monitoring	Runs the evaluation set and tracks production quality	Ragas, DeepEval, custom runners, observability tools	ML with SMEs

Product names are examples of what is commonly used, not recommendations. Choose by your cloud, security rules and team skills.

8.2 Request flow

1. The request arrives with the user's identity attached.

2. Guardrails check the input for scope, personal data and injection patterns.
3. The orchestrator classifies the task using the ontology and selects the workflow.
4. The workflow calls tools in order: the database or APIs for exact data, RAG for policy context, each with the user's permissions applied.
5. Code computes the numbers and the rules engine evaluates the rules.
6. The LLM drafts the explanation using only the structured results and cited passages.
7. Validators check numbers, citations, scope and format.
8. If checks pass, the answer returns with sources and rule IDs. If not, the system retries within limits or escalates to a human.
9. Inputs, versions, sources and outputs are logged.

8.3 Deployment options

Option	Description	Pros	Cons	Typical fit
Customer cloud account (single tenant)	Stack runs in the bank's own cloud tenancy	Data stays inside; easiest security approval	Operations per customer	Default for banks
On premises	Runs in the bank's data center, often with self-hosted models	Maximum control	Hardware cost, slower updates, possibly weaker models	Strict regulators
Vendor-hosted, dedicated	Vendor runs a separate environment per customer	Fast to start	Data-residency and vendor-risk review	Smaller lenders
Hybrid	Rules and orchestration inside the customer boundary, model reached through a private endpoint	Balance of control and capability	More integration work	Common compromise

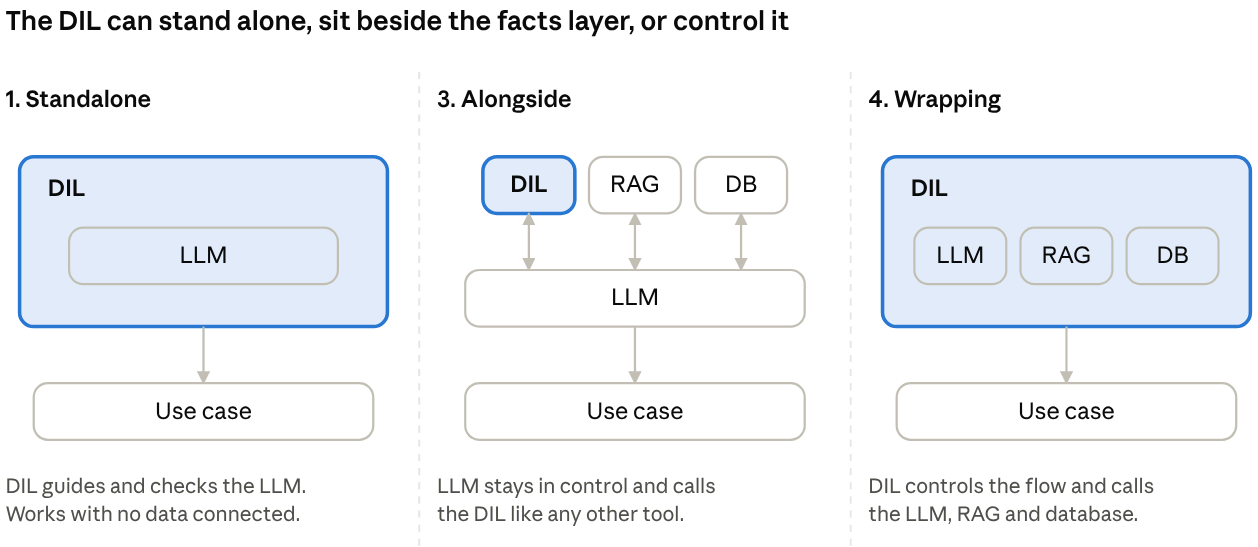
Option	Description	Pros	Cons	Typical fit
Multi-tenant SaaS	Shared platform with logical separation	Lowest cost, fastest updates	Hardest to approve for core bank data	Non-sensitive use cases

8.4 Design principles

- The LLM never makes the final rule decision or calculation; code does.
- Permissions follow the user and are enforced at retrieval and query time.
- Every output carries its sources, rule IDs and versions.
- Everything configurable lives outside the model (rules, workflows, glossary) and is versioned.
- The model layer is replaceable.
- When unsure, the system escalates instead of guessing.

9. Ways to integrate the DIL into the pipeline

There are seven practical ways to attach a DIL to an LLM pipeline. For regulated, rule-heavy work, wrapping the facts layer in policy-gated orchestration (pattern 4) gives the strongest control; the others are entry points, accelerators or special cases.



three core DIL placements · patterns 1, 3 and 4 of 7

The drawing shows the three core placements from the original scenario diagram: the DIL alone, beside RAG and the database, or controlling them.

9.1 The seven patterns

#	Pattern	How it works	Best when	Pros	Cons
1	Standalone	Rules, glossary, expert prompts and validators wrap the model; no company data is connected	Early pilots; drafting and review where the user supplies the facts	Fast and cheap; no data-access review	Cannot know company policy or customer facts
2	Data in the prompt	A small set of documents or records is placed in the context window together with DIL instructions and checks	One-off tasks; small, static knowledge	Simple; nothing to tune	Context limits and cost; no per-user access control; goes stale
3	Alongside	The LLM stays in control and calls the DIL as tools (rule checker, glossary lookup, validator) next to RAG and database tools	Teams with an existing LLM app and tools	Easy to add	The model may skip or misuse the tools, so rules are applied less reliably
4	Wrapping (policy-gated orchestration)	The DIL runs the workflow in code and calls the LLM only for specific steps; rules and validators gate every output	Regulated, rule-heavy tasks that need consistency	Strongest control, audit trail and consistency	More design up front; less flexible for open-ended questions
5	Smart retrieval	The DIL improves RAG before and after retrieval: expands terms, picks sources, adds filters for product, date and version, then validates retrieved text	Large policy libraries with many versions and near-duplicates	Better retrieval accuracy with modest change	Needs good metadata; helps only retrieval-heavy tasks

#	Pattern	How it works	Best when	Pros	Cons
6	DIL plus fine-tuned model	A model adapted to domain language and formats, with the DIL still enforcing rules	High volume with stable format; smaller or on-premises models	Shorter prompts, lower cost per task, consistent tone	Training effort; retraining when the model changes; does not replace rules
7	Agentic with live data and memory	Agents loop over tools, including live APIs and user memory; the DIL supplies policy, tool allowlists, stopping rules and checks	Investigative work such as AML alert triage	Handles complex, multi-step cases	Hardest to test; higher risk and cost; needs strict permissions and human gates

9.2 How to choose

Situation	Start with
Early pilot, no data access yet	Pattern 1, then 2
Existing LLM app, want quick rule improvements	Pattern 3
Regulated workflow with clear steps (default for banking)	Pattern 4
Large policy library with versions and near-duplicates	Pattern 5, combined with 4
High volume, stable format, cost pressure	Add pattern 6 to 4
Investigation-style work with many branches	Pattern 7 running inside the gates of 4

9.3 Pattern 4 in detail

Policy-gated orchestration is the recommended default because it separates three jobs that LLMs do unevenly: understanding text, calculating and deciding, and writing.

- **The LLM** reads documents, extracts fields, explains results and drafts text.
- **Code and the rules engine** calculate, compare and decide.
- **The orchestrator** fixes the order of steps, so a required check cannot be skipped.

- **Gates** sit at the end of steps: if a validator fails, the workflow retries within limits or escalates to a person.

A pilot can still start in pattern 1 or 2 to show value in days, then move the same rules and workflows into pattern 4 without rewriting them, because rules, glossary and workflows live outside the model.

9.4 Pattern 5 in detail

Plain RAG often retrieves the most similar passage, which may be an outdated circular or the wrong product's policy. Smart retrieval adds the domain knowledge that fixes this.

1. **Before retrieval:** identify the product, customer type and date from the question; expand abbreviations and synonyms using the ontology; choose which collections to search.
2. **During retrieval:** filter by metadata (product, effective date, version, jurisdiction); combine keyword and vector search; rerank.
3. **After retrieval:** drop superseded documents; check that retrieved passages actually apply to the case; attach source and effective date to each passage.

9.5 Combining the DIL with other ways of getting data to the model

Data approach	What the DIL adds
Long-context prompts	Decides what to include and in what order; checks the answer against rules; prevents irrelevant bulk
Hybrid search (keyword plus vector)	Adds filters and term expansion; verifies exact identifiers such as policy numbers
Knowledge graph and GraphRAG	The ontology can seed or constrain the graph; relationships guide which sources to query
Text-to-SQL	Restricts allowed tables and columns; supplies approved query templates for key metrics; validates results
Real-time data and APIs	Sets freshness rules and flags stale data; restricts which endpoints may be called
User memory and context	Decides what may be stored and reused; masks sensitive details

Data approach	What the DIL adds
Fine-tuning	Provides the training data, test cases and rules that the tuned model is checked against

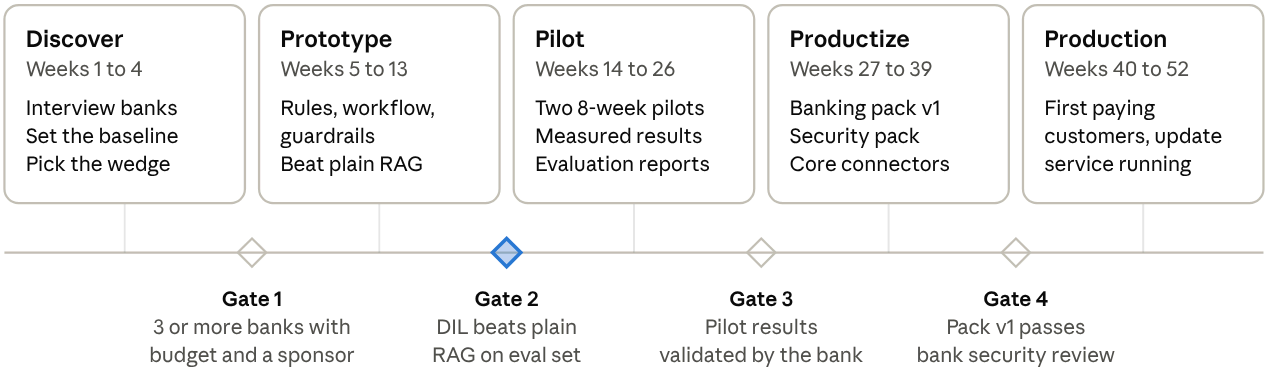
9.6 Recommended path

1. **Weeks 1 to 4:** pattern 1 and 2 for a fast demonstration using public rules and a few sample documents.
2. **Weeks 5 to 12:** pattern 4 for the pilot workflow, with pattern 5 added if retrieval quality is the main error source.
3. **After the pilot:** add pattern 6 only if the evaluation shows a style, cost or latency problem that prompting cannot fix; use pattern 7 only for tasks that need multi-step investigation, and keep human gates.

10. Build plan

Build in five phases over 12 months with a go or stop gate after each of the first four. The team is small, about 12 people at full strength, and about a third of the effort is domain expertise rather than engineering.

A gate at the end of each phase decides whether to continue



12-month build plan · 5 phases, 4 gates · phases drawn to equal width, not to scale

Gate 2, highlighted, is the make-or-break test: if the DIL does not clearly beat plain RAG on the evaluation set, the concept needs rework before any pilot money is spent.

10.1 Team

The headcount is an illustrative assumption for the full 12-month plan. Phase 1 can start with four or five people and add the rest as the work grows.

Role	People	Main responsibility
Product lead	1	Scope, customers, priorities, pricing
Domain experts (credit, compliance)	2	Rules, workflows, evaluation grading, regulatory review; may be part-time advisors
Rules and knowledge engineer	1	Turns policy into rules, glossary and ontology; maintains the repository
LLM and ML engineers	2	Workflows, prompts, retrieval, model gateway, evaluation tooling
Backend and platform engineers	2	Orchestration, APIs, deployment, access control
Data engineer	1	Document parsing, pipelines, connectors
Evaluation and quality engineer	1	Evaluation harness, regression gates, monitoring
Security and deployment engineer	1	Private deployments, security pack, audit logging
Implementation lead	1	Pilots, customer onboarding, feedback to product

10.2 Phase 1: Discover (weeks 1 to 4)

- **Objectives:** confirm the wedge, the buyer and the budget path; set a measurable baseline.
- **Activities:** customer interviews; collect sample documents and public rules; build the first 100 evaluation cases; stand up a plain-RAG baseline and score it.
- **Deliverables:** wedge definition, baseline scores, list of candidate design partners.
- **Gate 1:** three or more institutions with budget and an executive sponsor for the wedge.

10.3 Phase 2: Prototype (weeks 5 to 13)

- **Objectives:** show that the DIL beats plain RAG on the evaluation set.
- **Activities:** glossary and first 30 to 50 rules with tests; the first workflow with computed ratios, rule checks and citations; guardrails for personal data, scope, numbers and citations; expand the evaluation set to 300 cases including adversarial ones.
- **Deliverables:** working prototype in pattern 4, score report versus baseline, a security overview.
- **Gate 2:** the DIL beats the plain-RAG baseline by the margin agreed in advance.

10.4 Phase 3: Pilot (weeks 14 to 26)

- **Objectives:** prove value inside a real bank with the customer's own baseline.
- **Activities:** deploy inside the customer's boundary; connect two or three systems; run two 8-week pilots with weekly metrics; collect reviewer feedback and turn corrections into tests.
- **Deliverables:** pilot reports per customer, an updated rule library, a list of integration gaps.
- **Gate 3:** pilot results validated by the bank against the agreed targets, and a path to a paid production contract.

10.5 Phase 4: Productize (weeks 27 to 39)

- **Objectives:** turn repeated pilot work into a standard banking pack.
- **Activities:** package rule templates, workflows, evaluation set and deployment scripts; harden connectors for the most common systems; write the security and model-risk documentation pack; set up the regulatory update process.
- **Deliverables:** banking pack v1, deployment automation, documentation pack.
- **Gate 4:** the pack passes a bank's security review and a new deployment needs far less custom work than the first.

10.6 Phase 5: Production and scale (weeks 40 to 52)

- **Objectives:** first paying production customers and the first channel partner.
- **Activities:** production rollouts with staged release; training for implementation partners; marketplace listing; start the second workflow or a second function.

- **Deliverables:** two to three production customers, a running rule-update service, a channel agreement.

10.7 Build, buy or reuse

Buy or reuse the commodity plumbing; build the domain content, because that is where the value and the moat are.

Component	Choice	Reason
Document parsing	Reuse open-source or managed	Commodity, but quality matters for tables and scans
Vector store and retrieval	Reuse managed or open-source	Commodity
LLM models	Buy access; stay model-agnostic	Not a differentiator; avoids lock-in
Orchestration framework	Reuse	Commodity; keep workflow specs portable
Guardrail mechanics	Reuse frameworks; write the policies	The policies are the value
Rules engine	Reuse the engine; build the rules	The rules are the moat
Ontology and glossary	Build	Domain-specific
Expert workflows	Build	Domain-specific
Evaluation sets	Build with experts	Proof and moat
Connectors	Build the top few; partner for the rest	Needed early, but not unique
Observability and logging	Reuse	Commodity

10.8 Technology selection criteria

- Can it run inside a customer's cloud account or data center?

- Does it support per-user access control at query time?
- Can rules and workflows be exported in a portable, versioned format?
- Does it let you swap models without rewriting workflows?
- Is there a clear audit trail for every decision?
- What is the exit path if the vendor changes terms?

10.9 Common mistakes

- Building the chat interface first and the evaluation set last.
- Encoding rules as long prompts instead of testable checks.
- Letting the LLM do arithmetic or final decisions.
- Starting with every banking function at once.
- Treating regulatory updates as a one-time import.
- Skipping the plain-RAG control, so the DIL's added value is never proven.

11. Evaluation and quality

Evaluation is the part that turns the DIL from a promising demo into something a risk committee can approve. Build the evaluation set before the first workflow, run it on every change and report the score to the customer.

11.1 Building the evaluation set

1. **Collect real tasks.** Pull 300 to 500 real questions and cases from analyst work, support tickets and past files, with personal data removed.
2. **Tag each case.** Task type, product, rule or rules involved, difficulty and risk level.
3. **Write the expected result.** For factual questions, the correct answer and its source; for judgment tasks, acceptance criteria (must mention, must not mention, correct decision, correct rule ID).
4. **Add adversarial cases.** Superseded circulars, conflicting documents, requests to bypass controls, ambiguous inputs, prompt-injection text inside uploaded documents.
5. **Have two SMEs grade a sample** to measure how much experts agree with each other; disagreement shows which rules are unclear.
6. **Freeze a held-out portion** (around a fifth) that the team never tunes against, to detect overfitting.
7. **Version the set** and record the score history for every release.

11.2 Metrics

Metric	What it measures	How to compute
Factual accuracy	Claims match the source	Expert grading or reference-based scoring
Rule-application accuracy	Correct rule chosen and correct outcome	Compare with expected rule ID and decision
Numeric accuracy	Calculations correct	Automatic recomputation in code
Citation precision and recall	Claims supported by cited sources; important sources cited	Check each claim against its source
Refusal and escalation correctness	Refuses what it should, answers what it should	Labeled should-refuse and should-answer cases
Over-refusal rate	Valid requests wrongly blocked	Share of should-answer cases refused
Consistency	Same question gives equivalent answers	Run each case several times and compare
Latency and cost per task	Operational viability	Measured per workflow run
Reviewer acceptance	Output accepted by human reviewers with few edits	Production sampling

Automated graders using an LLM can speed up scoring, but they must be calibrated against expert grades on a sample, and experts should always grade the high-risk cases.

11.3 Release process

- **Regression gate.** No change to a rule, prompt, model or retrieval setting ships if scores on protected categories drop below agreed thresholds.
- **Shadow mode.** New versions run alongside the live version and are compared before cutover.
- **Staged rollout.** One team first, then wider, with a rollback plan.

- **Model changes.** Switching or upgrading the underlying LLM triggers the full evaluation, because behavior can shift.

11.4 Human review loop

Place human review where risk is highest and keep it measurable. Reviewer edits and rejections feed back as new test cases and rule fixes. Track which rules produce the most corrections; they are the ones to clarify first.

11.5 Production monitoring

- Sample a fixed share of outputs for expert review each week.
- Track escalation rate, refusal rate, reviewer edit distance and complaint reports.
- Alert on drift: a sudden change in retrieval sources, rule hit rates or answer length can signal a problem.
- Log rule and workflow versions with every output so any answer can be reproduced.

12. Security, compliance and governance

In banking, security and governance decide whether the product is bought at all. Design them in from the first week: access control at retrieval time, data kept inside the customer's boundary, full audit trails and a documented model-risk process.

12.1 Security requirements

Area	Requirement	How the DIL meets it
Access control	Users see only what they are entitled to	Permissions enforced at retrieval and query time using the user's identity, never the model's own access
Data residency	Data stays in approved regions or on premises	Deploy inside the customer's cloud account or data center; no data leaves the boundary
PII protection	Sensitive fields are masked or tokenized	Masking before prompts and logs; separate secure store for originals

Area	Requirement	How the DIL meets it
Prompt injection	Documents or users cannot override rules	Treat retrieved text as data; rules and system instructions live outside the model's editable context; validate tool calls
Least privilege	Tools and connectors have narrow scopes	Per-workflow tool allowlists; read-only by default
Secrets and keys	No credentials in prompts or code	Central secrets manager; short-lived tokens
Logging	Every action is traceable	Immutable logs with user, inputs (masked), rule versions, sources, outputs
Vendor risk	Third-party models are assessed	Contractual limits on data use and retention; option for private or self-hosted models

12.2 Governance and model risk management

Banks typically already run a model risk management process, and AI systems enter it. Expect to supply:

- A model inventory entry describing purpose, owner, inputs, outputs and limits.
- Documentation of the rule set, workflows, guardrails and their owners.
- Validation evidence: evaluation scores, adversarial test results and bias or fairness checks where decisions affect customers.
- A change-management policy: who may change a rule, how it is tested and who approves it.
- A monitoring plan with thresholds and incident procedures.
- An exit plan: what happens if the model vendor changes terms or the model is withdrawn.

12.3 Regulatory landscape to check

The applicable regimes depend on the customer's country. The table lists categories to review with legal counsel, not legal advice.

Category	Examples to review
Banking supervision and model risk	Central bank guidance on outsourcing, IT risk and model risk; Basel principles
Financial crime	AML and counter-terrorist-financing rules, FATF recommendations, sanctions regimes
Data protection	GDPR in Europe, India's Digital Personal Data Protection Act, local banking secrecy laws
AI regulation	The EU AI Act and national AI guidance; credit scoring is commonly treated as high-risk
Consumer protection	Fair treatment, disclosure and complaint-handling rules
Cloud and outsourcing	Rules for material outsourcing and cloud use by regulated entities

Regulations change frequently; verify the current text and timelines for each target market before committing to a design or a launch date.

12.4 Human oversight

- High-impact decisions (approving credit, closing accounts, filing reports) stay with a human, with the DIL preparing and checking the work.
- Every output shows its sources, the rule IDs applied and a confidence or completeness indicator.
- Users can flag wrong outputs in one click, and flags go into the evaluation backlog.

12.5 Responsible AI checks

- **Fairness.** Where outputs influence decisions about people, test results across customer segments and document the findings.
- **Explainability.** Rule IDs and cited sources give reviewers and customers a reason for each outcome.
- **Transparency.** Customers and staff know when AI assisted, as required by local rules and bank policy.

13. Go-to-market

Sell a measurable outcome in one narrow workflow to a small number of design-partner banks, then expand through integrators and cloud marketplaces. Do not launch as "AI for banking"; launch as, for example, "faster, auditable credit file review for mid-size lenders".

13.1 Positioning

Element	Statement
Target customer	Mid-size banks, NBFCs and regional lenders that have document-heavy credit and compliance work but small AI teams
Problem	Analysts spend long hours on repetitive review; generic AI tools give fluent but unreliable answers; risk teams will not approve what they cannot measure
Promise	Faster review with rule-checked, cited, auditable output, proven against your own evaluation set
Proof	Pilot results against the customer's own baseline, and a published accuracy report on the evaluation set
Differentiator	Expert-curated rules and workflows, local regulation coverage and measured accuracy, not just a chat interface

13.2 Ideal customer profile and buyers

Role	Cares about	What they need to see
Head of credit or operations (economic buyer)	Turnaround time, cost per file, quality	Baseline vs pilot results
Chief risk or compliance officer (gatekeeper)	Control, audit, regulation	Rule traceability, evaluation scores, oversight design
CIO or CISO (gatekeeper)	Security, integration, vendor risk	Deployment inside their boundary, access control, logs

Role	Cares about	What they need to see
Analysts (users)	Less rework, trustworthy output	Time saved, easy escalation, no extra clicking
Procurement	Price, terms, exit options	Clear pricing, data terms, exit plan

13.3 Phased plan

1. **Discovery (months 0 to 2).** Interview 15 to 20 people across 8 to 10 institutions to validate the use case, current cost per case and budget path. Re-score the domain table from real answers.
2. **Design partners (months 2 to 6).** Sign 2 to 3 banks to co-build. Offer reduced pricing and priority support in return for access to experts, anonymized baselines and a reference. Their experts help create the evaluation set.
3. **Pilots with proof (months 4 to 9).** Run 8-week pilots with agreed metrics and a plain-RAG control. Publish an accuracy and time-saving report per pilot with the customer's consent.
4. **Productize (months 6 to 12).** Turn the repeated work from partners into a standard domain pack: rules templates, workflows, evaluation set, connectors, deployment scripts.
5. **Channels (months 9 to 18).** Partner with system integrators, core-banking vendors and cloud marketplaces that already have bank relationships and approved procurement paths.
6. **Expand (months 12 to 24).** Add adjacent banking workflows, then insurance, reusing the architecture.

13.4 Channels

Channel	Advantage	Watch-out
Direct sales	Control, learning, margin	Slow; needs senior sellers who know banking
System integrators	Existing trust and projects	Revenue share; integrators may build rivals

Channel	Advantage	Watch-out
Cloud marketplaces	Use of committed cloud spend; simpler procurement	Fees; competition from native cloud offerings
Core-banking and workflow vendors	Embedded in daily tools	Dependence on their roadmap
Consultancies and audit firms	Credibility on governance	Long cycles; may want to white-label

13.5 Sales motion

- **Land:** a paid pilot priced inside the budget a department head can approve without a board decision.
- **Prove:** weekly dashboards during the pilot showing time saved, accuracy and escalations.
- **Expand:** more teams, more products, more regions; then adjacent functions.
- **Renew:** rule-update subscription tied to regulatory change keeps the layer current and the contract recurring.

13.6 Messaging that works with risk-averse buyers

- Lead with control and evidence, not with the model: "every answer shows the rule and the source".
- Offer deployment inside the customer's cloud account or data center.
- Show failure cases openly: what the system refuses, escalates and gets wrong, and how it is measured.
- Compare against the customer's baseline and a plain-RAG control, not against competitors' marketing claims.

13.7 Metrics for the go-to-market itself

Stage	Metric
Discovery	Interviews completed; share of institutions with budget and an executive sponsor

Stage	Metric
Design partners	Signed partners; experts′ hours committed per week
Pilots	Pilot-to-contract conversion; improvement vs baseline; evaluation score
Revenue	Annual recurring revenue; implementation share of revenue; net revenue retention
Efficiency	Sales cycle length; implementation weeks per customer; gross margin

14. Business model and unit economics

The layer earns money for the vendor through a domain-pack license, a rule-update subscription and implementation fees, and earns money for the customer by cutting analyst time, speeding up decisions and reducing compliance errors. Price against the customer′s value, not against your build cost.

All numbers in this section are **illustrative assumptions** to show the arithmetic. Replace them with figures from your own pilot and customer baselines.

14.1 Revenue models

Model	How it works	Pros	Cons	Fit
Annual domain-pack license	Fixed yearly fee for the banking pack, including updates and support	Predictable revenue; simple for buyers	Needs proof of value before purchase	Core model
Rule-update subscription	Recurring fee for keeping rules current with regulatory change	Aligns with ongoing cost of expert work	Can be seen as optional; bundle with the license	Add-on or bundled

Model	How it works	Pros	Cons	Fit
Implementation and integration fee	One-time fee for connectors, data setup, evaluation set and training	Covers delivery cost; filters serious buyers	Slows deals if too high	Always
Usage-based	Fee per case, workflow run or document processed	Easy to start; scales with value	Revenue varies; buyers fear surprise bills	Good for volume workflows
Per-seat	Fee per analyst user	Familiar	Value is in cases, not seats; encourages seat-sharing	Secondary
Outcome-based	Share of measured savings	Strong alignment	Hard to measure and audit; slow cash	Selective, with trusted partners
Managed service	You operate the system in the customer's cloud	Higher margin and retention	Needs operations staff and strong security	Larger customers

Recommended structure: annual license plus rule-update subscription plus a one-time implementation fee, with a usage band above an agreed volume.

14.2 Illustrative customer value

The example is a mid-size lender reviewing credit files.

Item	Assumption or result
Files reviewed per year	12,000
Analyst time per file today	3.0 hours
Fully loaded analyst cost	USD 25 per hour
Cost of manual review today	USD 900,000 per year (12,000 × 3.0 × 25)

Item	Assumption or result
Assumed time saving from the DIL	30%
Annual saving	USD 270,000 (10,800 hours × 25)

The 30% saving is an assumption for the arithmetic only. The pilot must measure the real figure; if it is lower, price and scope change.

14.3 Illustrative pricing and customer return

Item	Year 1	Year 2 and after
Domain-pack license	USD 90,000	USD 90,000
Rule-update subscription	USD 20,000	USD 20,000
Implementation (one time)	USD 60,000	none
Total customer cost	**USD 170,000**	**USD 110,000**
Customer saving	USD 270,000	USD 270,000
Net benefit	**USD 100,000**	**USD 160,000**

At these assumptions the customer's year-1 net benefit is about 59% of what it spends (100,000 ÷ 170,000), and payback is about 7.6 months if savings start immediately (170,000 ÷ 22,500 per month). Real payback will be longer because savings ramp up as users adopt the tool.

14.4 Illustrative vendor economics

Item	Year 1	Year 2 and after
Revenue	USD 170,000	USD 110,000
Cost to serve recurring revenue (cloud, model usage, support, rule upkeep), assumed 35% of recurring revenue	USD 38,500	USD 38,500
Implementation delivery cost, assumed	USD 40,000	none
Gross profit	**USD 91,500**	**USD 71,500**

Item	Year 1	Year 2 and after
Gross margin	about 54%	65%

If the team costs USD 1.8 million per year (assumed 12 people at USD 150,000 fully loaded), recurring gross profit alone needs roughly 25 customers to cover it (1,800,000 ÷ 71,500), before sales and marketing. That is why productizing, reusing rule packs across customers and keeping implementation effort low matter more than any single deal.

14.5 Cost structure

Cost	Driver	How to control
Domain experts	Rules, workflows, evaluation sets, regulatory updates	Reuse across customers; part-time expert network; templates
Engineering	Orchestration, connectors, validators, deployment	Use managed RAG and guardrail components
Model inference	Calls per task, model size	Smaller models for checks, caching, routing
Cloud and infrastructure	Per-customer private deployments	Infrastructure as code; standard stack
Security and compliance	Audits, certifications, penetration tests	Plan certifications early; reuse evidence
Sales and customer success	Long enterprise cycles	Design-partner references; channel partners

14.6 Levers that improve the economics

- Raise the share of rules and workflows that are shared across customers in the same country.
- Standardize connectors for the three or four most common core-banking and loan-origination systems.
- Offer the rule-update subscription as the main recurring value, since regulations keep changing.
- Use channel partners for implementation so your own delivery cost stays low.

15. Competitive landscape and moat

No single product today delivers a complete, ready-to-use banking DIL; competitors cover pieces of it. The opening exists, but the pipeline code is not the moat. The expert-curated rules, the proprietary evaluation data and the trust built with regulated customers are.

The named vendors below are examples from general knowledge and are not a complete or current market map. Verify the current offerings before using this section externally.

15.1 Who covers which piece

Layer of the market	Examples	What they provide	What is usually missing
Managed RAG and search platforms	AWS Bedrock Knowledge Bases, Azure AI Search with Azure OpenAI, Google Vertex AI Search, Databricks, Snowflake Cortex, Elastic, Cohere, Vectara	Ingestion, retrieval, reranking, citations	Domain rules, workflows and expert evaluation
Open-source frameworks	LlamaIndex, LangChain and LangGraph, Haystack, RAGFlow, Dify	Building blocks for pipelines and agents	All domain content; production hardening is on you
Guardrail tooling	NeMo Guardrails, Guardrails AI, cloud guardrail services	Policy enforcement mechanics	The rules themselves and how an expert would apply them
Domain-specific AI products	In finance: research and analysis tools such as AlphaSense and Hebbia, banking assistants such as Kasisto; in legal: Harvey, CoCounsel; in healthcare: Abridge, OpenEvidence	Workflows for specific content types	Not built to run on a bank's own arbitrary internal policies and data

Layer of the market	Examples	What they provide	What is usually missing
Model vendors' industry packs	Industry-specific offerings from large model providers, such as Claude for Financial Services	General domain workflows and connectors	Local regulation, the customer's own policies and deep integration
RegTech vendors	KYC, AML and sanctions-screening providers	Deep rules and data in a narrow function	Conversational and document-reasoning layer
Systems integrators and consultancies	Large global and regional firms	Customization and delivery	Reusable product; margins depend on hours
In-house teams at large banks	Internal AI platform groups	Full control	Time, scarce experts; most do not want to maintain domain content alone

15.2 Where a new DIL can win

- **Local regulation and practice.** Global vendors cover major regimes first; depth in one country's rules, forms and practices is a real advantage.
- **Customer-specific rules.** Each bank's credit policy is different; a product that ingests and operationalizes the customer's own policy beats a generic one.
- **Proof.** A published, customer-validated evaluation score is rare and persuasive.
- **Neutral across models and clouds.** Banks often want to avoid lock-in to one model or cloud; a layer that moves with them is attractive.

15.3 Sources of defensibility

Moat	How it is built	How durable
Expert-curated rule and workflow library	Years of SME input, tested against real cases	High, but only if kept current
Proprietary evaluation data	Anonymized, expert-graded cases from many customers	High; hard to copy, improves with each deployment

Moat	How it is built	How durable
Regulatory update service	A team that tracks and encodes changes	Medium to high; requires ongoing cost
Trust and references	Production deployments in regulated banks, completed model-risk reviews	High; slow to earn
Integrations	Certified connectors to common banking systems	Medium
Ecosystem	Channel partners trained on your pack	Medium
Pipeline code	Orchestration and glue	Low; anyone can write it

Customer data must never be used across customers without explicit contractual permission; the evaluation-data moat depends on anonymization and consent being handled properly.

15.4 Threats

- Hyperscalers and model vendors bundling domain packs at low or zero marginal price.
- Large banks building in-house and hiring away your experts.
- Better base models reducing the benefit of workflows that were compensating for model weakness. Rules and guardrails stay valuable; some prompting workarounds will not.
- Standardization of agent and rules formats making packs easier to copy.

15.5 Response

Concentrate on what a hyperscaler will not do: country-specific rules, customer-specific policy operationalization, regulatory update service and a hands-on pilot-to-production path. Make the DIL run on top of the customers' chosen model and cloud, including the vendors listed above, rather than against them.

16. Risks and mitigations

The four risks most likely to stop the venture are slow bank procurement, the cost of keeping expert content current, competition from bundled packs, and failure to prove measurable accuracy. Each has a specific early warning sign and a response.

Likelihood and impact are qualitative planning judgments (High, Medium, Low), to be revisited as evidence comes in.

#	Risk	Likelihood	Impact	Early warning	Mitigation
1	Procurement and security reviews take far longer than planned	High	High	Pilot stuck in review for over 3 months	Deploy inside the customer's boundary; pre-build a security pack; target budgets that need no board approval; use cloud marketplaces
2	Expert content is expensive and goes stale	High	High	Rule review dates overdue; rising reviewer corrections	Owner and review date on every rule; regulatory feed; shared rule library across customers; part-time expert network
3	Pilot cannot show a measurable gain over plain RAG	Medium	High	Control and DIL scores are close on the evaluation set	Choose the use case where rules dominate; build the evaluation set first; stop or pivot early if the gap is small
4	Hyperscalers and model vendors bundle similar packs	High	Medium	Competitor announcements in your wedge	Differentiate on local rules, customer policy and evidence; stay model- and cloud-neutral
5	A wrong rule or guardrail failure causes a compliance incident	Low to medium	High	Rule hit rates change suddenly; reviewer overrides spike	Unit tests per rule; human approval on high-risk steps; staged rollout; incident process

#	Risk	Likelihood	Impact	Early warning	Mitigation
6	Data access and quality block the pilot	Medium	Medium	Required data not available by week 3	Define data needs in the pilot contract; use a sandbox copy; start with documents the bank can share
7	Over-restriction makes users abandon the tool	Medium	Medium	Low usage after week 2; high refusal rate	Track refusal rate; tune with users; keep an easy escalation path
8	Prompt injection or data leakage	Medium	High	Red-team findings; odd tool calls in logs	Treat retrieved text as data; narrow tool scopes; filter outputs; regular red-team exercises
9	Model vendor changes terms, pricing or behavior	Medium	Medium	Evaluation scores shift after a model update	Abstract the model layer; keep alternates; run evaluation on every change
10	Customer-specific work prevents productization	High	Medium	Each deployment needs more than half custom work	Limit custom scope in contracts; extract repeated work into the standard pack each quarter
11	Regulatory change in AI rules for finance	Medium	Medium	New guidance or consultation papers	Track regulators; design for transparency and human oversight from the start
12	Key-expert dependency	Medium	High	One person owns most of the rule library	Document rules with sources; pair experts; build an advisory panel

16.1 Kill criteria

Decide in advance what evidence would stop or redirect the project:

- After 3 discovery rounds, fewer than 3 institutions have budget and an executive sponsor for the wedge use case.
- In two pilots, the DIL does not beat the plain-RAG control on the evaluation set by the margin agreed in advance.

- Implementation effort per customer stays above the level that allows the target gross margin after the third deployment.

17. Roadmap and next steps

The next 90 days should answer three questions with evidence: do banks have budget for this wedge, can a small rule-and-workflow layer beat plain RAG on a real evaluation set, and which integration pattern fits the first design partner. The 12-month plan then turns the answers into a repeatable product.

17.1 Decisions needed now

- [ ] Confirm the name and one-line positioning (Section 3).
- [ ] Confirm the first domain and wedge use case (Sections 6 and 7).
- [ ] Choose the target country or region for rules, since rule content is local.
- [ ] Decide the build, buy and partner split for plumbing (Section 10).
- [ ] Decide funding and team size for the first 12 months, including part-time expert access.
- [ ] Decide the deployment stance: inside customer cloud or data center by default.

17.2 First 30 days

1. Interview 8 to 10 candidate banks and lenders; capture cost per file, current tools, budget path and blockers.
2. Recruit two or three senior credit or compliance experts as advisors, paid or equity-based.
3. Collect 50 sample documents and policies (public ones are enough to start) and map the rule sources.
4. Build the first version of the evaluation set: 100 cases for one workflow.
5. Stand up a minimal pipeline using managed RAG and a rules engine; baseline plain RAG against the 100 cases.

17.3 Days 31 to 60

1. Implement the terminology list and first 30 to 50 rules for the wedge workflow, each with a source and test.
2. Build the first workflow with computed ratios, rule checks, citations and escalation.

3. Add guardrails: PII masking, scope control, numeric and citation validators.
4. Compare the DIL against the plain-RAG baseline on the evaluation set and record the gap.
5. Prepare a security overview and deployment guide for bank reviewers.

17.4 Days 61 to 90

1. Present results to 3 to 5 prospective design partners and agree one or two paid pilots.
2. Expand the evaluation set to 300 cases, including adversarial ones.
3. Define pilot success metrics and the baseline measurement method with each partner.
4. Draft pricing and contract terms for pilots, including data and security terms.
5. Decide go, adjust or stop against the kill criteria in Section 16.1.

17.5 Months 4 to 12

Period	Goal	Key outputs
Months 4 to 6	Run pilots	Two 8-week pilots with measured results; evaluation score reports
Months 7 to 9	Productize	Standard banking pack v1: rules templates, workflows, evaluation set, connectors, deployment scripts
Months 10 to 12	First production deployments and channel start	Two to three production customers; one integrator or marketplace listing; regulatory update process running

17.6 Success checkpoints

Checkpoint	Evidence required
Day 90	Three or more institutions confirm budget and sponsor; the DIL beats plain RAG on the evaluation set by the agreed margin
Month 6	At least one pilot complete with customer-validated results

Checkpoint	Evidence required
Month 12	Paying production customers; implementation effort per customer falling; rule-update process on schedule

18. Glossary and appendix

This appendix defines the terms used in the document and gives three concrete templates a team can copy: a rule, a workflow and an evaluation case. All values in the templates are invented for illustration and are not real policy.

18.1 Glossary

Term	Meaning
Domain Intelligence Layer (DIL)	Domain rules, terminology, workflows, guardrails and tests that make an LLM reason like a specialist
LLM	Large language model, the text-generating model at the center of the system
RAG	Retrieval-augmented generation: fetch relevant documents first, then have the model answer using them
Embedding	A numeric representation of text used to find similar passages
Vector database	A store that finds passages by embedding similarity
Reranking	A second step that reorders retrieved passages by relevance to the question
Hybrid search	Combining keyword search and vector search
GraphRAG	Retrieval that follows relationships in a knowledge graph
Text-to-SQL	Having the model write a database query from a question

Term	Meaning
Fine-tuning	Further training a model on your data to change its behavior or style
Ontology	A structured vocabulary of concepts and the relationships between them
Semantic layer	Business definitions of metrics and terms used by analytics tools
Guardrail	A check that blocks or corrects unsafe or out-of-scope inputs and outputs
Orchestration	Code that decides the order of steps, tool calls and checks in a task
Prompt injection	Text that tries to override the system's instructions
Evaluation set	Test cases with expert-approved expected results
SME	Subject-matter expert
Model risk management	A bank's process for approving, validating and monitoring models
KYC	Know your customer: verifying customer identity and risk
AML	Anti-money-laundering controls
PEP	Politically exposed person, a higher-risk customer category
NPA	Non-performing asset: a loan whose payments are overdue beyond a defined period
CASA	Current account and savings account deposits
LTV	Loan-to-value ratio
DSCR	Debt service coverage ratio

18.2 Sample rule (illustrative)

```
id: CR-LTV-004
title: Maximum loan-to-value for secured term loans
applies_to: [secured_term_loan]
condition: ltv_ratio <= 0.75
on_fail: refer_to_senior_credit
source: Credit Policy Manual, section 4.3 (example reference)
effective_from: 2026-04-01
review_by: 2027-04-01
owner: Head of Credit Risk
tests:
  - input: {ltv_ratio: 0.70}
    expect: pass
  - input: {ltv_ratio: 0.80}
    expect: refer_to_senior_credit
```

Every rule has an ID, a machine-checkable condition, a source, dates, an owner and tests. The model never decides pass or fail; it reads the result and explains it.

18.3 Sample workflow specification (illustrative)

```
workflow: credit_file_review
version: 0.3
inputs: [borrower_file, product_type]
steps:
  - id: classify
    action: identify_borrower_type
    uses: [ontology]
  - id: extract
    action: extract_financials
    uses: [llm, validator.totals_reconcile]
  - id: compute
    action: compute_ratios
    uses: [code]
  - id: rules
    action: evaluate_rules
    uses: [rules_engine]
    on_fail: escalate_to_reviewer
  - id: retrieve
    action: fetch_policy_context
    uses: [rag]
    filters: [product_type, effective_date]
```

```
  - id: draft
    action: write_memo
    uses: [llm]
    requires: [citations]
  - id: validate
    action: check_draft
    uses: [validator.numbers, validator.citations, guardrail.scope]
outputs: [memo, rule_results, sources, escalations]
logging: [rule_versions, sources, reviewer_decision]
```

18.4 Sample evaluation case (illustrative)

```
{
  "id": "EV-0142",
  "task": "credit_file_review",
  "risk": "high",
  "input": "Borrower file with LTV of 0.80 on a secured term loan",
  "must_include": ["rule CR-LTV-004", "referral to senior credit"],
  "must_not_include": ["approve"],
  "expected_decision": "refer_to_senior_credit",
  "adversarial": false
}
```

18.5 Sources and limits of this document

This document was written from general knowledge of enterprise AI practice and banking operations, without live market research or legal review. Vendor names are examples. Figures in Sections 6, 14 and 16 are planning judgments or illustrative assumptions. Regulations named in Section 12 should be checked against current official texts for each target country before any design or launch decision.